Propuesta Inicial de Entidades de Base de Datos - MiDoc

A continuación, se presenta la estructura base para el modelo relacional de MiDoc, enfocado en cubrir las funcionalidades esenciales del producto mínimo viable (autenticación, gestión de citas, control de expedientes y suscripciones).

1. Usuario

Almacena las credenciales globales y la identidad de cualquier persona en la plataforma.

Campo	**Tipo de Dato**	**Descripción / Clave**
id	UUID / INT	Clave primaria
email	VARCHAR(150)	Correo electrónico (Único)
password_hash	VARCHAR(255)	Contraseña encriptada
rol	ENUM	Roles: 'PACIENTE', 'DOCTOR', 'CLINICA'
fecha_registro	TIMESTAMP	Fecha de creación de la cuenta

2. PerfilDoctor

Contiene la información profesional y comercial de los doctores o clínicas.

Campo	**Tipo de Dato**	**Descripción / Clave**
id	UUID / INT	Clave primaria
usuario_id	UUID / INT	Clave foránea -> Usuario(id)
nombre_completo	VARCHAR(150)	Nombre o razón social
especialidad	VARCHAR(100)	Área médica principal
ubicacion	VARCHAR(255)	Dirección o coordenadas del consultorio
precio_consulta	DECIMAL(10,2)	Costo por consulta

3. Cita

Gestiona las agendas y los estados de las consultas entre pacientes y doctores.

Campo	**Tipo de Dato**	**Descripción / Clave**
id	UUID / INT	Clave primaria
paciente_id	UUID / INT	Clave foránea -> Usuario(id)
doctor_id	UUID / INT	Clave foránea -> PerfilDoctor(id)
fecha_hora	DATETIME	Fecha y hora programada
estado	ENUM	Estado: 'PENDIENTE', 'CONFIRMADA', 'CANCELADA', 'COMPLETADA'

4. ConsentimientoExpediente

Controla qué profesionales de la salud tienen permiso para visualizar el expediente médico del paciente.

Campo	Tipo de Dato	Descripción / Clave
id	UUID / INT	Clave primaria
paciente_id	UUID / INT	Clave foránea -> Usuario(id)
doctor_id	UUID / INT	Clave foránea -> PerfilDoctor(id)
estado_acceso	BOOLEAN	true (Autorizado) / false (Revocado)
fecha_actualizacion	TIMESTAMP	Última modificación del permiso

5. Receta

Almacena el registro médico generado durante o después de una consulta.

Campo	Tipo de Dato	Descripción / Clave
id	UUID / INT	Clave primaria
cita_id	UUID / INT	Clave foránea -> Cita(id)
diagnostico	TEXT	Diagnóstico emitido por el doctor
tratamiento	TEXT	Indicaciones y medicamentos recetados
documento_url	VARCHAR(500)	Enlace al archivo digital de la receta (si aplica)

6. Suscripcion

Registra el estado del pago mensual de los doctores o clínicas para el uso del panel de gestión.

Campo	Tipo de Dato	Descripción / Clave
id	UUID / INT	Clave primaria
doctor_id	UUID / INT	Clave foránea -> PerfilDoctor(id)
estado	ENUM	Estado: 'ACTIVA', 'INACTIVA', 'PENDIENTE'
monto	DECIMAL(10,2)	Valor fijado ($20.00 USD)
fecha_inicio	DATE	Inicio del periodo pagado
fecha_fin	DATE	Vencimiento del periodo

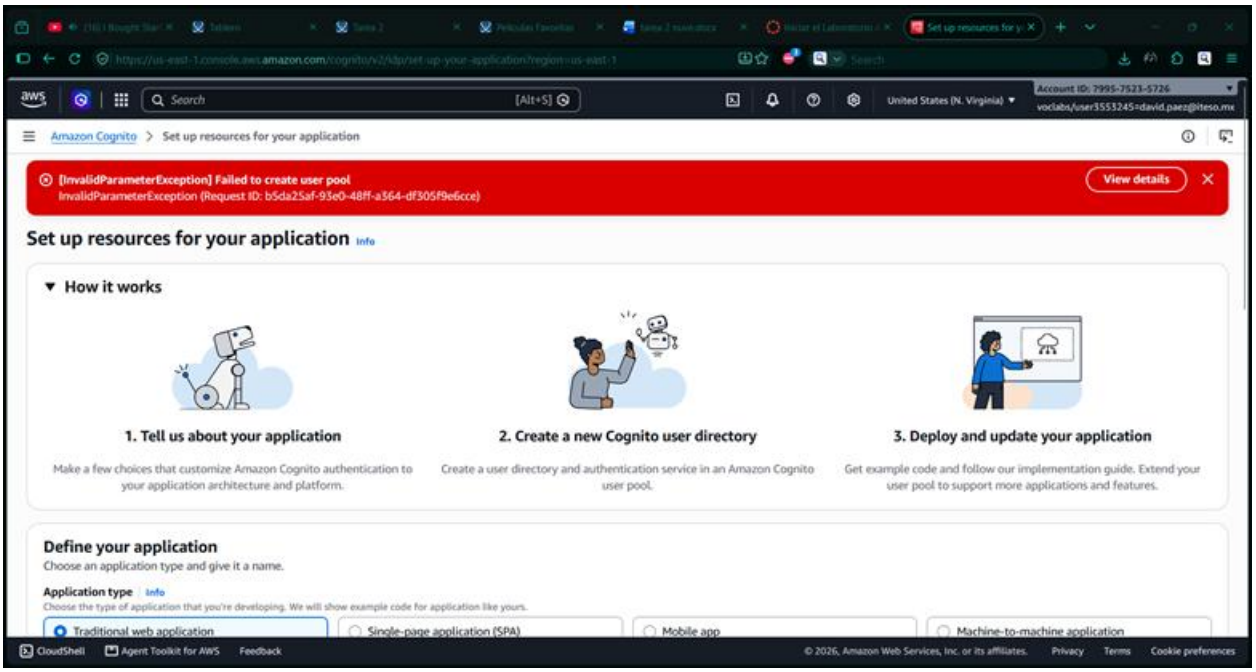
Justificación del Diseño

Esta propuesta adopta un modelo relacional estandarizado porque garantiza la integridad referencial y la consistencia ACID necesaria para gestionar información sensible como agendas, transacciones de suscripción y permisos explícitos de expedientes médicos. Al separar la entidad genérica de usuario de los perfiles

específicos y desacoplar los permisos mediante una tabla de consentimiento (ConsentimientoExpediente), logramos una arquitectura de datos escalable, ágil de consultar y totalmente independiente de cualquier proveedor o infraestructura subyacente.

¿Cómo sustituir Cognito?

Debido a la falta de privilegios que da el laboratorio de AWS, no podemos hacer User pools y aunque la opción de Identity pools sí se pueden crear, la verdad es que no se puede conectar con la aplicación por consecuencia de los privilegios que ofrece AWS Academy



Solución

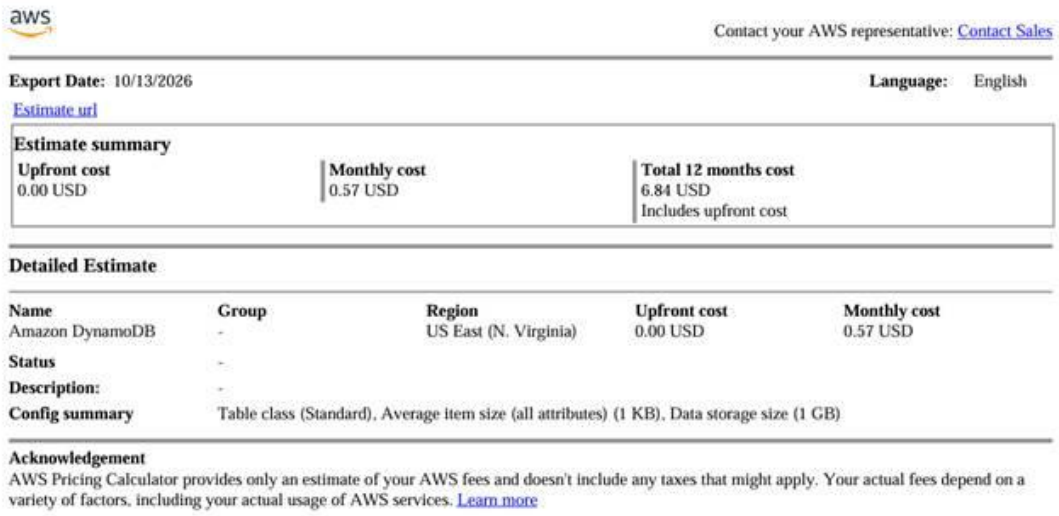
La propuesta que traemos es la de utilizar Secrets Manager para ser nuestro quinto servicio. Ya que Mi Doc maneja información sensible, esto supone que los JWT así como las credenciales no deben de quedar como variables de entorno. AWS Secrets Manager resuelve esto porque cifra los datos y restringe su lectura por medio de permisos IAM, así solo las funciones que nosotros autoricemos pueden acceder a la información.

Otro punto a favor de Secrets Manager es que centraliza la gestión de los secretos y permite rotarlos, es decir, cambiar la llave o las credenciales sin modificar ni volver a desplegar Código. Además, que la integración con Lambda es directa.

Finalmente, nuestro equipo ya está familiarizado con este servicio gracias a las actividades que hemos hecho en clase.

¿Es viable integrar DynamoDB a los servicios del proyecto, sabiendo que ya tenemos RDS con PostgreSQL?

Investigando me di cuenta de que integrar DynamoDB es viable, siempre que complemente a PostgreSQL y no se integre solo porque sí. A diferencia de Cognito, está disponible en el Learner Lab, y la prueba de ello es la práctica de los logs donde lo usamos, así que no necesitaríamos de una cuenta externa. También, cobra solo por uso, y haciendo un experimento de una cotización con 300,000 escrituras, un millón de lecturas y 1 GB al mes, salió que costaría $0.57 USD (AWS, s. f.-a), por lo que es muy rentable considerando los créditos que tenemos del Lab.



aws

Contact your AWS representative: Contact Sales

Export Date: 10/13/2026 **Language:** English

Estimate url

Estimate summary

Upfront cost	Monthly cost	Total 12 months cost
0.00 USD	0.57 USD	6.84 USD Includes upfront cost

Detailed Estimate

Name	Group	Region	Upfront cost	Monthly cost
Amazon DynamoDB	-	US East (N. Virginia)	0.00 USD	0.57 USD
Status	-			
Description:	-			
Config summary	Table class (Standard), Average item size (all attributes) (1 KB), Data storage size (1 GB)			

Acknowledgement

AWS Pricing Calculator provides only an estimate of your AWS fees and doesn't include any taxes that might apply. Your actual fees depend on a variety of factors, including your actual usage of AWS services. Learn more

Además, es serverless como Lambda y cifra todas sus tablas en reposo, algo importante para datos de salud como los del proyecto (AWS, s. f.-c).

Este patrón de usar en un mismo sistema la base de datos que mejor se ajusta a cada tipo de dato se llama persistencia políglota (Fowler, 2011). AWS lo recomienda para arquitecturas con Lambda y API Gateway como la nuestra (AWS, s. f.-b), y se aplica en sistemas grandes. De ejemplos está Amazon, que movió sus cargas transaccionales de alta consistencia a Aurora y RDS, y sus servicios de baja latencia a DynamoDB (Barr, 2019), y también Netflix, que mantuvo los pagos en MySQL porque ocupan transacciones ACID, y movió los datos de suscriptores y el historial de cobros a Cassandra (Vlaovic et al., 2016).

Haciéndonos una idea para MiDoc, lo que necesita relaciones, transacciones o restricciones iría a la base relacional, mientras que lo que solo crece y se consulta siempre por la misma llave iría a la no relacional, algo así (no es un modelado

completo/exacto y es meramente hipotético, la propuesta del modelado detallado la desarrolló Emmanuel en su investigación):

- Relacional (RDS PostgreSQL): usuarios y perfiles, citas, suscripciones y pagos, consentimientos de acceso al expediente, consultas, recetas e inventario.
- No relacional (DynamoDB): bitácora de accesos al expediente (quién lo vio y cuándo), notificaciones y mensajes paciente-doctor, en una sola tabla con llaves como PACIENTE#id y ACCESO#fecha.

La principal desventaja es que DynamoDB no puede hacer JOINs, por lo que conviene meterlo de forma acotada, empezando por la bitácora de accesos.

Conclusión y Decisión

Como conclusión, creo que su inclusión es claramente viable por las razones comentadas antes y por lo que nos comentaba la maestra de que nuestro proyecto es algo complejo en cuanto a datos; pero aun así, yo lo añadiría como un posible sexto servicio a implementar si el proyecto se desarrolla a buen ritmo y sobra tiempo, para así tener una arquitectura más robusta, ya que considero más importante la inclusión de Secrets Manager como el quinto servicio por las razones dadas por David en su investigación.

Referencias

Amazon Web Services. (s. f.-a). Amazon DynamoDB pricing. Recuperado el 13 de octubre de 2026, de https://calculator.aws/#/createCalculator/DynamoDB

Amazon Web Services. (s. f.-b). Database-per-service pattern. AWS Prescriptive Guidance. https://docs.aws.amazon.com/prescriptive-guidance/latest/modernization-data-persistence/database-per-service.html

Amazon Web Services. (s. f.-c). DynamoDB encryption at rest usage notes. Amazon DynamoDB Developer Guide. https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/encryption.usagenotes.html

Barr, J. (2019, 15 de octubre). Migration complete – Amazon's consumer business just turned off its final Oracle database. AWS News Blog. https://aws.amazon.com/blogs/aws/migration-complete-amazons-consumer-business-just-turned-off-its-final-oracle-database/

Fowler, M. (2011, 16 de noviembre). Polyglot persistence. martinfowler.com. https://www.martinfowler.com/bliki/PolyglotPersistence.html

Vlaovic, S., Pilani, R., Parulekar, S., y Handa, S. (2016, 21 de junio). Netflix billing migration to AWS. Netflix Technology Blog. https://netflixtechblog.com/netflix-billing-migration-to-aws-451fba085a4